

Meu Plantão

Guia para entender os arquivos do projeto e a lógica do código

Versão 1.0 | 30/05/2026

Como usar este guia

Este PDF foi escrito para alguém que vai olhar o repositório pela primeira vez e precisa entender onde cada responsabilidade mora: layout global, providers, sidebar, páginas, componentes de dados, estados locais e regras visuais. A ideia é que você consiga navegar pelos arquivos e saber o que ler primeiro, o que cada bloco controla e como o sistema se organiza antes de integrar uma API real.

Ponto de partida recomendado:

1) `app/layout.tsx` 2) `components/sidebar/sidebar.tsx` 3) componentes das telas

1. Mapa do projeto

A estrutura já mostra a separação entre roteamento, shell global, componentes de UI e páginas funcionais. Os seeds e estados locais estão concentrados dentro de cada domínio para facilitar troca por API.

1.1 Raiz do projeto

Arquivo	O que faz	Como ler o código
app/	Pasta de rotas do Next App Router. Cada subpasta dentro de app/pages vira uma tela.	Comece aqui para entender como as páginas são montadas.
components/	Componentes compartilhados e componentes de domínio, como dashboard, plantões, equipe, relatórios e configurações.	É onde mora a maior parte da UI e da lógica local.
lib/	Funções utilitárias pequenas, como helpers de classes.	Use quando precisar de helpers puros.
public/	Arquivos estáticos, como imagens e logo.	É onde ficam assets públicos usados em Image.
docs/	Saída dos PDFs gerados.	Os guias finais são salvos aqui.

1.2 Ordem de leitura recomendada

- Leia `app/layout.tsx` primeiro para entender providers, fontes globais e a base da aplicação.
- Depois leia `components/sidebar/sidebar.tsx` para entender navegação e idioma.
- Em seguida, leia a página que você quer consumir: dashboard, plantões, equipe, relatórios ou configurações.
- Se a tela depender de dados, abra também o arquivo `\*-data.ts` correspondente para ver o formato dos objetos.

Dica de navegação

O padrão do projeto é simples: a rota em app/pages só aponta para um componente em components. A lógica de verdade fica no componente de domínio, não no arquivo de rota.

2. Arquivos centrais e como lê-los

Esses são os arquivos que ditam o comportamento global. Eles ajudam a entender tema, idioma, shell visual e estrutura geral da tela.

2.1 app/layout.tsx

Esse arquivo é a raiz do app. Ele importa as fontes, aplica os providers e define a estrutura HTML e body de todas as telas. No Next.js App Router, ele é o lugar ideal para injetar tema e idioma globais.

```
export default function RootLayout({ children }: Readonly<{ children: React.ReactNode; }>) {
  return (
    <html lang="en" suppressHydrationWarning className={cn(...)}>
      <body className="min-h-full flex flex-col bg-background text-foreground">
        <ThemeProvider attribute="class" defaultTheme="system" enableSystem disableTransitionOnChange>
          <LanguageProvider>{children}</LanguageProvider>
        </ThemeProvider>
      </body>
    </html>
  );
}
```

- O `ThemeProvider` controla o dark mode via `next-themes`.
- O `LanguageProvider` guarda o idioma e atualiza `document.documentElement.lang`.
- A classe `bg-background text-foreground` garante que os tokens globais do CSS controlem visualmente o app inteiro.

2.2 components/theme-provider.tsx

É um wrapper fino em cima do provider do `next-themes`. Ele existe para isolar a dependência e deixar a árvore do app limpa.

```
export function ThemeProvider({ children, ...props }: React.ComponentProps<typeof NextThemesProvider>) {
  return <NextThemesProvider {...props}>{children}</NextThemesProvider>;
}
```

- O valor padrão é `system`, então o tema acompanha o sistema do usuário.
- O `disableTransitionOnChange` evita animações estranhas ao alternar tema.

2.3 components/language-provider.tsx

Esse provider é responsável pelo idioma global. Ele lê o `localStorage`, expõe `language` e `setLanguage` e escreve o idioma no documento para refletir no HTML.

```
const [language, setLanguageState] = React.useState<AppLanguage>("pt-BR");

React.useEffect(() => {
  setLanguageState(getInitialLanguage());
}, []);

React.useEffect(() => {
  document.documentElement.lang = language;
  window.localStorage.setItem(storageKey, language);
}, [language]);
```

- Use esse provider como fonte única da linguagem global.
- Se a tela quiser mudar o idioma, deve chamar `setLanguage` e não manipular `localStorage` diretamente.

2.4 components/sidebar/sidebar.tsx

A sidebar é o shell de navegação. Ela combina navegação, troca de idioma e tema, e exibe o contexto de administração.

```
const navItems = [
  { label: "Dashboard", href: "/pages/dashboard", icon: faChartLine },
  { label: "Plantões", href: "/pages/plantoes", icon: faCalendarDays },
  { label: "Equipe", href: "/pages/equipe", icon: faUserDoctor },
  { label: "Relatórios", href: "/pages/relatorios", icon: faClipboardList },
  { label: "Configurações", href: "/pages/configuracoes", icon: faGear },
];
```

- O item ativo usa `usePathname()` para realçar a rota atual.
- Os rótulos trocam de idioma de acordo com o `language` global.
- O `ModeToggle` fica no rodapé da sidebar para alternar o tema.

3. Páginas e componentes de domínio

Cada tela principal é organizada em um componente de domínio com os estados, filtros e ações daquela área. É aqui que entram os seeds e os contratos de dados.

3.1 Dashboard

A dashboard é o resumo operacional. O arquivo `components/dashboard/page.tsx` monta a faixa de filtros, o resumo rápido, o gráfico semanal, a lista de atenção e a tabela recente.

```
const maxValue = Math.max(...shiftSummary.map((item) => item.value), 1);

return (
  <section className="space-y-6">
    <FilterBar />
    <SummaryCard ... />
    <WeeklyChart />
    <AttentionList />
    <RecentShiftsTable />
  </section>
);
```

- O dashboard consome `shiftSummary`, `weeklyVolume`, `urgentItems` e `recentShifts` vindos de `dashboard-data.ts`.
- Os cards usam `toneStyles` para variar cor, ícone e barra de progresso.
- O componente foi desenhado para ser lido rapidamente pelo usuário, então os blocos são independentes.

3.2 components/dashboard/dashboard-data.ts

Esse arquivo é o coração dos dados da dashboard. Ele define os tons visuais, os cards principais, o volume semanal e a lista urgente.

```
export const shiftSummary: SummaryItem[] = [
  {
    label: "Aprovados",
    value: 24,
    tone: "emerald",
    icon: faCircleCheck,
    description: "Solicitações liberadas e prontas para acontecer.",
    comparison: "+8% vs semana passada",
    comparisonTone: "positive",
  },
];
```

- Se o backend mudar, tente manter esse shape dos objetos.
- É nesse arquivo que também mora a paleta de tons dos cards.

3.3 components/plantoes/plantoes.tsx

É a tela operacional de plantões. Ela concentra busca, filtro, tabela, modal de edição e modal de confirmação para aprovação/recusa.

```
const [search, setSearch] = useState("");
const [status, setStatus] = useState<PlantaoStatus | "Todos">("Todos");
const [items, setItems] = useState<PlantaoItem[]>(plantaoRows);
const [selectedPlantao, setSelectedPlantao] = useState<PlantaoItem | null>(null);
```

- A busca filtra por várias colunas concatenadas.
- A aprovação e recusa passam por `pendingAction` para abrir confirmação antes de alterar o status.
- A edição trabalha em duas camadas: `selectedPlantao` e `draftPlantao`.

3.4 components/plantoes/plantoes-data.ts

Aqui ficam os tipos e os seeds da tela de plantões. Esse arquivo é o contrato dos dados que a tabela e o modal esperam.

```
export type PlantaoItem = {
  id: string;
  unidade: string;
  profissional: string;
  especialidade: string;
  periodo: string;
  solicitante: string;
  status: PlantaoStatus;
  observacao: string;
};
```

- Use isso como base para uma API futura.
- Os status fechados ajudam na renderização de badges e botões.

3.5 components/relatorios/page.tsx

Relatórios é a visão histórica. A página soma totais, filtra os registros e prepara a exportação em PDF com os dados já filtrados.

```
const summary = useMemo(() => {
  const total = relatoriosSeed.length;
  const completos = relatoriosSeed.filter((row) => row.status === "Completo").length;
  const emAberto = relatoriosSeed.filter((row) => row.status === "Em aberto").length;
  const receitaTotal = relatoriosSeed
    .filter((row) => row.status === "Completo")
    .reduce((sum, row) => sum + row.valorGerado, 0);

  return { total, completos, emAberto, receitaTotal };
}, []);
```

- O PDF exportado depende da lista filtrada, não da lista bruta.
- Esse padrão é bom para integrar API sem mudar a lógica de exibição.

3.6 components/relatorios/relatorios-data.ts

Esse arquivo define o contrato do histórico de relatórios e os estilos de status. O front-end lê daqui o formato dos dados e a cor do badge.

```
export type RelatorioPlantao = {
  id: string;
  unidade: string;
  profissional: string;
  concluidoPor: string;
  data: string;
  valorGerado: number;
  status: RelatorioStatus;
};
```

3.7 components/configuracoes/page.tsx

Configurações funciona como um formulário com rascunho e salvamento. O usuário altera campos localmente e só aplica as mudanças ao salvar.

```
const [settings, setSettings] = useState<ConfiguracoesState>(configuracoesIniciais);
const [savedSettings, setSavedSettings] = useState<ConfiguracoesState>(configuracoesIniciais);

function saveSettings() {
  setSavedSettings(settings);
  setLanguage(settings.idioma === "English" ? "en" : "pt-BR");
  setTheme(settings.darkMode ? "system" : "light");
}
```

- Esse é o melhor exemplo de estado local + confirmação explícita.

- O idioma global e o tema só mudam quando o usuário salva.

4. Como o projeto se guia por arquivos

Esse é o mapa mental para um novo desenvolvedor navegar sem se perder.

- `app/layout.tsx` mostra a fundação da aplicação.
- `components/sidebar/sidebar.tsx` mostra a navegação e as dependências globais.
- Cada página em `components/<domínio>/page.tsx` mostra a composição da tela.
- Os arquivos `\*-data.ts` mostram o formato esperado dos dados.
- Os subcomponentes como `\*-filters.tsx`, `\*-table.tsx`, `\*-modal.tsx` mostram a lógica visual e interativa por parte da interface.

Regra prática

Se você quer entender uma tela, leia primeiro o arquivo page.tsx daquela pasta. Depois leia o data.ts correspondente. Em seguida, abra os componentes menores na ordem: filtros, tabela, modal e ações.

4.1 Onde o front-end deve conectar a API

Arquivo	O que faz	Como ler o código
Dashboard	Resumo, gráfico e lista de urgência.	Os objetos devem ter o mesmo shape dos seeds.
Plantões	Lista com filtros e ações de status.	Mantém Plantaoltem, PlantaoStatus e os fluxos de modal.
Equipe	CRUD de usuários.	Mantém EquipeMember e EquipeRole.
Relatórios	Histórico e exportação.	Mantém RelatorioPlantao e os status.
Configurações	Preferências do sistema.	Mantém ConfiguracoesState.

5. Regras visuais e de manutenção

Essas regras ajudam a manter o projeto coeso quando o front-end passar a consumir dados reais.

- O projeto trabalha com superfície escura, bordas retas e contraste forte.
- Os estados de sucesso, alerta e erro usam verde, âmbar e rosa/vermelho.
- Evite reintroduzir azul forte nas superfícies principais; ele deve ficar só se for realmente um dado semântico.
- Quando um componente tiver fonte de dados local, deixe claro se isso é mock, seed ou contrato de API.
- Se houver loading, preserve a composição visual para não quebrar a leitura da tela.

Resumo final

O jeito mais seguro de evoluir esse projeto é preservar o contrato dos arquivos de dados, ler a página de domínio como ponto inicial e fazer o backend devolver exatamente o mesmo shape que os componentes já usam hoje.